

অপারেটিং সিস্টেমের ভূমিকা
(Introduction to Operating Systems)
প্রফেসর চেস্টার রেবেরো

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ
(Department of Computer Science and Engineering)
ভারতীয় প্রৌদ্যোগিকী সংস্থান, মাদ্রাজ
(Indian Institute of Technology, Madras)

সপ্তাহ – 01 বক্তৃতা- 18 CPU সিডিউলিং
(CPU Scheduling)

নমস্কার। এই বক্তৃতাতে, আমরা সিপিইউ সিডিউলিং অ্যালগোরিদমগুলো দেখব।

(স্লাইড সময় : 00:22 দেখুন)

আমরা অপারেটিং সিস্টেমগুলিতে দেখেছি যে সিডিউলার একটি নির্দিষ্ট প্রসেসকে ready কিউ থেকে বেছে নেবে এবং প্রসেসটি প্রসেসরে চালানোর জন্য নির্ধারিত হবে। এখন যে প্রশ্নটি আমরা বিশ্লেষণ করতে যাচ্ছি তা কি এই সিডিউলার পরবর্তী প্রসেসকে কীভাবে বেছে নেবে? মূলত, নির্ধারিত প্রসেসের ready কিউ থেকে সিপিইউতে চালানোর জন্য কীভাবে সিডিউলার একটি প্রসেসকে পছন্দ করবে?

(স্লাইড সময় : 00:53 দেখুন)

এখন এটা বিশ্লেষণ করার জন্য, আমরা প্রথমে একটি প্রসেসের এক্সিকিউশন পর্যায়গুলি দেখবো। এখন যে কোনও প্রোগ্রামের এক্সিকিউশনের দুইটি পর্যায় রয়েছে; এক যখন এটি আসলে ইন্সট্রাকশন সম্পাদন করে যা একটি CPU বাস্ট হিসাবে পরিচিত হয়, অন্যটি যখন এটি একটি I/O ব্লক করা হয় বা কোন অপারেশন না করে, তাই এই বিশেষ ক্ষেত্রে, CPU টি কোনও কাজ করে না। সুতরাং, আমরা সময়ের সাথে সাথে দেখি, একটি নির্দিষ্ট প্রসেসের কিছু CPU বাস্ট রয়েছে যার মধ্যে প্রসেসরের ইন্সট্রাকশনগুলি এক্সিকিউট হয়, তারপর এটি একটি নিষ্ক্রিয় সময় থাকবে যার মধ্যে এটি একটি I / O অপারেশনের জন্য অপেক্ষা করছে। তারপর সিপিইউ সময়ের একটি বাস্ট তৈরি হবে। এইভাবে, CPU- বাস্ট এবং নিষ্ক্রিয় সময় মধ্যে এই অন্তর্বর্তী সবসময় একটি অপারেশনের জন্য অপেক্ষা করে।

(স্লাইড সময় : 01:51 দেখুন)

অতএব, এক্সিকিউশনগুলির ক্ষেত্রে এইগুলির উপর ভিত্তি করে প্রসেসগুলিকে দুটি প্রকারের মধ্যে শ্রেণিভুক্ত করতে পারে। একটি হল I / O বাউন্ড প্রসেস, অন্যটি CPU বাউন্ড প্রসেস। আমরা কেন এই I / O বাউন্ড এবং CPU বাউন্ড প্রসেসের মধ্যে পার্থক্য করি? মূলত, এটি একটি সিডিউলিং এর দৃষ্টিকোণ থেকে, আমরা I / O বাউন্ড প্রসেসকে একটি উচ্চতর অগ্রাধিকার দিতে চাই যা দিয়ে তারা CPU গুলি বরাদ্দ করে থাকে। মূলত, আমরা চাই যে I / O বাউন্ড প্রসেস CPU- র সীমিত প্রসেসগুলির তুলনায় CPU এর জন্য কম সময় অপেক্ষা করে। সুতরাং, কেন এই প্রয়োজন, আমরা একটি উদাহরণ নিতে পারি। সুতরাং ধরা যাক আমরা একটি ওয়ার্ড প্রসেসর যেমন নোটপ্যাড বা Vim ব্যবহার করছি এবং আমরা এই I / O বাউন্ড প্রসেসকে একটি খুব কম অগ্রাধিকার প্রদান করছি। এখন, ধরি কোনও ব্যবহারকারী একটি কী চাপছেন, কারণ এটির কম গুরুত্ব রয়েছে, এটি খুব বেশি সময় ধরে CPU না পায় এবং তাই, স্ক্রিনে ব্যবহারকারীর দ্বারা যে কী চাপা হয় তার আগে এটি কিছু সময় নেয়। সুতরাং, ব্যবহারকারীর জন্য এটি বেশ অস্বস্তিকর হতে পারে, তাই আমরা I / O বাউন্ড প্রসেস যেমন, ওয়ার্ড প্রসেসরকে উচ্চতর অগ্রাধিকার প্রদান করতে চাই, যার সাহায্যে এটি সিপিইউ পাবে, যাতে ব্যবহারকারীর ইন্টারঅ্যাকশনটি সিপিইউতে আরও আরামদায়ক হয়। অন্য দিকে, যদি আপনি CPU- র বাউন্ড প্রসেস দেখেন, আমরা এটিকে একটি নিম্ন অগ্রাধিকার দিতে পারি। উদাহরণস্বরূপ, যদি আপনি এই অ্যাপ্লিকেশনগুলোর মধ্যে একটি গ্রহণ করেন, যেমন, CPU- র বাউন্ড অ্যাপ্লিকেশনগুলি আমাদের বলে যে 'gcc' একটি প্রোগ্রাম কম্পাইল করছে এবং আমাদের বলে যে আপনি একটি বড় প্রোগ্রাম সংকলন করছেন যা 5 মিনিট সময় নেয়। এখন এটি এই ব্যবহারকারীকে বেশি প্রভাবিত করবে না যদি সেই নির্দিষ্ট প্রোগ্রামটি কম্পাইল করতে সময় 5 মিনিট থেকে 5.5 মিনিট পর্যন্ত বৃদ্ধি পায়। সুতরাং, একটি CPU বাউন্ড প্রসেস কম অগ্রাধিকারের সঙ্গে কাজ করতে পারে। I / O বাউন্ড এবং CPU বাউন্ড এর মধ্যে এই শ্রেণীবিভাগটি একটি অনমনীয় শ্রেণিবিন্যাস নয়, এটি একটি প্রসেসর একটি সময় I / O বাউন্ড প্রসেস হতে পারে, এবং কিছু সময় পরে এটি একটি CPU বাউন্ড প্রসেসর মত আচরণ করতে পারে। সুতরাং, উদাহরণ হিসাবে, একটি প্রসেস যা উভয় I / O বাউন্ড এবং CPU- বাউন্ড এর মত আচরণ করে, আপনি উদাহরণস্বরূপ মাইক্রোসফট এক্সেল নিতে পারেন। যখন আমরা আসলে Excel মধ্যে বিভিন্ন কোষে তথ্য প্রবেশ করাই, এটি I / O বাউন্ড প্রসেস হিসাবে কাজ করে। সুতরাং, এটি ছোট CPU বাস্ট এবং একটি বৃহৎ সময়ের I / O চক্রের সাথে I / O বাউন্ড প্রসেসের মত আচরণ করে। অন্য দিকে যখন আপনি আসলে প্রবেশকৃত ডাটাগুলির কিছু পরিসংখ্যান কম্পিউটিং করছেন, তখন এক্সেল একটি CPU বাউন্ড প্রসেসের মত আচরণ করবে, যেখানে CPU কার্যকলাপের একটি বড় অংশ বা প্রকৃতপক্ষে নির্দিষ্ট ডেটাতে কাজ করার সময় নেওয়া হয়।

(স্লাইড সময় : 04:38 দেখুন)

এখন, আসুন আমরা প্রশ্ন করি যে সিপিইউ সিডিউলার কীভাবে ready প্রসেসের কিউ থেকে সিপিইউতে এক্সিকিউট করা পরবর্তী প্রসেসকে কীভাবে নির্বাচন করা উচিত। এই বিকল্পটি তৈরি করার এমন অনেক উপায় থাকতে পারে; মূলত, অনেক সিপিইউ নির্ধারণ এলগরিদম থাকতে পারে যা কিউতে দেখবে এবং একটি নির্দিষ্ট সিদ্ধান্ত নেয়।

(স্লাইড সময় : 05:05 দেখুন)

তাই এই বিভিন্ন সিডিউলিং আলগোরিদিম তুলনা করার জন্য, অপারেটিং সিস্টেম টেক্সট বই বা অপারেটিং সিস্টেম গবেষণা বিভিন্ন রকমের সিডিউলিং মানদণ্ড সংজ্ঞায়িত করে। তাই, এই মানদণ্ডটি আসলে তাদের প্রত্যেকের সুবিধা এবং অসুবিধা দেখতে বিভিন্ন সময় সিডিউলিং অ্যালগরিদমের তুলনা করতে ব্যবহার করা যেতে পারে। সুতরাং আসুন আমরা এই প্রতিটি সিডিউলিং এর মানদণ্ড এক এক করে দেখে যাই। প্রথম সিডিউলিং টি হল সিপিইউ সদ্যবহার। সিডিউলিং অ্যালগরিদম এইভাবে ডিজাইন করা উচিত যাতে CPU ব্যবহার সর্বাধিক হয়। অন্য কথায়, যতটা সম্ভব ন্যূনতম সময় হিসাবে CPUটি নিষ্ক্রিয় থাকে। পরের মাপকাঠিটি, আমরা দেখবো ফ্রপুট। মূলত, সিডিউলিং অ্যালগরিদমগুলি প্রতি ইউনিটে সিডিউলিং যতটা সম্ভব প্রসেস সম্পন্ন করার চেষ্টা করবে। একটি তৃতীয় উপায় হল টার্নঅ্যারাউণ্ড টাইম এবং এই মানদণ্ডটি একক প্রসেসর দৃষ্টিকোণ থেকে বিবেচনা করা হয়। অতএব, একটি প্রসেস শুরু থেকে শেষ পর্যন্ত জন্য সময় হল টার্নঅ্যারাউণ্ড টাইম। চতুর্থ মাপকাঠি হল রেসপন্স টাইম। সুতরাং, এই বিন্দু থেকে নেওয়া সময় হিসাবে সংজ্ঞায়িত করা হয় যে প্রসেস যখন ready কিউ থেকে running স্টেটে প্রবেশ করে, যে তাত্ক্ষণিক সময় থেকে প্রসেস ready কিউতে আসে আর যে সময় পর্যন্ত CPU সংশ্লিষ্ট প্রসেসের ইন্সট্রাকসন এক্সিকিউট করা শুরু করে। আরেকটি মানদণ্ড হল waiting টাইম। এখন এই মানদণ্ডটি প্রস্তুত হয় একটি প্রসেস দ্বারা গ্রহীত সিডিউলিং এর উপর ভিত্তি করে। এখন আমরা জানি যে প্রসেস ready থেকে run করার জন্য প্রসেসকে ready কিউতে উপস্থিত থাকা প্রয়োজন এবং এটি যে তারা খুব দীর্ঘ সময় ready কিউতে অপেক্ষা করে না। সুতরাং, সিডিউলিং অ্যালগরিদমগুলি এমনভাবে ডিজাইন করা যেতে পারে যে রেডি কিউ এর waiting টাইম বা গড় waiting টাইম কমিয়ে আনা হয়। চূড়ান্ত মানদণ্ড আমরা এখন দেখব তা হল ফেয়ারনেস। সিডিউলারটি অবশ্যই নিশ্চিত হওয়া উচিত যে প্রতিটি প্রসেস নির্দিষ্ট কিছু নীতির উপর ভিত্তি করে CPU-র যথাযথ অংশ দেওয়া হয়। সুতরাং, এটা এমন কিছু না হওয়া উচিত যা কিছু প্রসেসের ক্ষেত্রে, যেমন, CPU এর 90 শতাংশ বলে মনে হয়, অন্য সব প্রসেসগুলি কেবল CPU এর 10 শতাংশ পায়। সুতরাং, একটি অপারেটিং সিস্টেমের জন্য একটি সিডিউলিং অ্যালগরিদম ডিজাইন করার সময় এই সমস্ত মানদণ্ড বিবেচনা করা প্রয়োজন। একটি একক সিডিউলিং অ্যালগরিদম একযোগে এই সমস্ত মানদণ্ড পূরণ করতে সক্ষম হবে না। অতএব, এই মানদণ্ডের একটি উপসেট পূরণের জন্য সিডিউলিং অ্যালগরিদমগুলি ডিজাইন করা হয়েছে। উদাহরণস্বরূপ, যদি আপনি বাস্তব সিডিউলিং অপারেটিং সিস্টেম বিবেচনা করেন, সেই সিস্টেমের জন্য

সিডিউলিং অ্যালগরিদমটি উদাহরণস্বরূপ ন্যূনতম প্রতিক্রিয়া সময়ের জন্য ডিজাইন করা হবে; অন্যান্য কারণ যেমন সিপিইউ ব্যবহার এবং থ্রুপুট মাধ্যমিক গুরুত্ব হতে পারে। অন্যদিকে, লিনাক্সের মত ডেস্কটপ অপারেটিং সিস্টেমটি নিখুঁততার জন্য ডিজাইন করা হবে, যাতে CPU বা সিস্টেমে চলমান সমস্ত অ্যাপ্লিকেশনগুলিকে CPU-র যথাযথ অংশ দেওয়া হয়। প্রতিক্রিয়া সময় যেমন মাপদণ্ড সেই দৃষ্টিকোণ থেকে কম গুরুত্বপূর্ণ হতে পারে। সুতরাং, আমরা এখন থেকে একাধিক সিডিউলিং অ্যালগরিদমগুলোর মধ্যে সহজতমটি থেকে শুরু করি যেটা হল ফার্স্ট কাম ফার্স্ট সার্ভ সিডিউলিং অ্যালগরিদম এবং আরও জটিল সিডিউলিং অ্যালগরিদমগুলিতে যেতে আমরা এগিয়ে যাই।

(স্লাইড সময় : 08:42 দেখুন)

তাহলে আসুন প্রথমে দেখা যাক প্রথমে FCFS সিডিউলিং অ্যালগরিদম দেখা যাক। এই ক্ষেত্রে মৌলিক প্রকল্প হলো যে CPU-র অনুরোধ করা প্রথম প্রসেসটি CPU তে বরাদ্দ করা হবে, রেডি কিউয়ের প্রথম প্রসেসটি CPU তে বরাদ্দ করা হয়। সুতরাং, এটি একটি অস্বত্প্রনোদিত সিডিউলিং অ্যালগরিদম যা এর মানে হল যে একবার একবার বরাদ্দকৃত প্রসেসটি সম্পূর্ণ না হওয়া পর্যন্ত CPU-এ এক্সিকিউট করতে থাকবে।

(স্লাইড সময় : 09:19 দেখুন)

আসুন আমরা একটি উদাহরণ দিয়ে এটি দেখি। ধরি আমাদের চারটি চলমান প্রসেসের সাথে আমাদের একটি সিস্টেম আছে; প্রসেসগুলি লেবেল P 1, P 2, P 3, P 4 এবং তাদের আগমনের সিডিউলিং রয়েছে, তাই আগমনের সিডিউলিং দ্বিতীয় কলামে উপস্থিত থাকে। তাই আগমনের সময় এই প্রসেস রেডি কিউতে প্রবেশ করার সময় থেকে সংজ্ঞায়িত করা হয়। সুতরাং, এই বিশেষভাবে খুব সহজ উদাহরণের জন্য, তাই আমরা বিবেচনা করব যে সমস্ত প্রসেসগুলি একই সময়ে রেডি কিউতে উপস্থিত হয় আর সেটা হল 0 সময়। তৃতীয় কলামটি CPU বাস্টের সময়, তাই এটি প্রত্যেকটি প্রসেসের জন্য CPU বাস্টের পরিমাণ দেয়। উদাহরণস্বরূপ, P1 এর 7 চক্রের একটি CPU বাস্ট রয়েছে; পি 2 এর 4 চক্র একটি বাস্ট আছে। এইভাবে, এই বিশেষ টেবিলে আমাদের চারটি প্রসেস আছে, যা সব পরপর রেডি কিউতে একই সময় 0 তে আসে এবং তাদের বিভিন্ন CPU বাস্টের টাইম থাকে, উদাহরণস্বরূপ P1 এর 7 চক্র, P 2 - 4 চক্র, P 3 - 2 চক্র এবং পি 4 - 5 চক্র। এখন, আমরা দেখব কিভাবে এই চারটি প্রসেস CPU-এ সিডিউলিং হয় বা কিভাবে এই চারটি প্রসেস CPU-কে বরাদ্দ করে। যেহেতু, এই সব প্রসেস একই সময়ে পৌঁছায়, সিডিউলারের আসলে নির্দিষ্ট কোনও পছন্দ থাকে না। সুতরাং, সে এলোমেলোভাবে একটি নির্দিষ্ট ক্রম বাছাই করবে। উদাহরণস্বরূপ, আসুন আমরা দেখি যে সিডিউলার P 1 চালানোর জন্য নেয়, তাই P 1 এর 7 চক্রের জন্য রান করে এবং এটি সম্পন্ন হলে, সিডিউলার P 2 এবং P 2 যা 4 চক্রের জন্য রান করে। পি 2 তার বাস্ট সমাপ্ত পরে, এবং পি 3 2 চক্র জন্য CPU মধ্যে এক্সিকিউট হয়; এবং তারপর পরিশেষে, P 4 5 CPU চক্রের জন্য চালানো হয়। সুতরাং, এটি একটি ন্যাট (Gantt) চার্ট দ্বারা প্রতিনিধিত্ব করা হয়। তাই,

একটি ন্যাট চার্ট হল একটি অনুভূমিক বার চিত্র যা 1917-এ হেনরি এল ন্যাট একটি উৎপাদনকারী যন্ত্র হিসেবে তৈরী করেন যিনি একজন আমেরিকান ইঞ্জিনিয়ার এবং সামাজিক বিজ্ঞানী ছিলেন। সুতরাং, মূলত একটি Gantt চার্ট আমাদের এখানে বিভিন্ন ব্লকের মত আছে এবং প্রতিটি ব্লক একটি এক্সিকিউশন চক্র বর্ণনা করে। সুতরাং, উদাহরণস্বরূপ, P 1 7 চক্রের জন্য সঞ্চালিত হয়, তাই এটি 7 ব্লক - 1, 2, 3, 4, 5, 6, 7, তারপর P 2 4 চক্রের জন্য এক্সিকিউট হয়, তাই এটি 4 ব্লকের মত দেওয়া হয়। তারপর P 3 2 চক্রের জন্য সঞ্চালিত হয়, তাই এটি 2 ব্লক দেওয়া হয় এবং পরিশেষে, P 4 5 চক্রের জন্য সঞ্চালিত হয়, তাই এটি 5 ব্লক দেওয়া হয়। সুতরাং, আমরা এই বিশেষ ক্ষেত্রে গড় waiting টাইম গণনা করতে পারে। আমরা দেখি যে প্রসেস P 1 তাত্ক্ষণিকভাবে রেডি কিউতে প্রবেশ করে এবং অবিলম্বে CPU এ এক্সিকিউট করে। সুতরাং, এটিকে সব সময়ে অপেক্ষা করতে হবে না। দ্বিতীয় প্রসেস P 2 ও 0 তে আসে, যা এই বিন্দুতে আছে কিন্তু P 1 এক্সিকিউটের পরেই চালানো হয় 7 সাইকেলের পরে; এইভাবে, P 2 প্রসেসের wait টাইম হয় 7। অনুরূপভাবে, প্রসেস P 3 যা 0 তম চক্রে প্রবেশ করে প্রসেসটি P1 এবং P 2 সমাপ্ত হওয়ার পরেই চালানো হয়। অন্য কথায়, এটিকে 11 চক্র অপেক্ষা করতে হবে এবং একইভাবে চতুর্থ প্রসেসকে 13 টি চক্রের জন্য অপেক্ষা করতে হবে। অতএব, এই বিশেষ ক্ষেত্রে গড় waiting টাইম 7.75 চক্র হয়। এখন, আমরা একটি অন্য সিডিউলিং নির্ধারণের মানদণ্ড দেখতে পারি, যা গড় রেসপন্স টাইম। সুতরাং, এই ক্ষেত্রে গড় রেসপন্স টাইম গড় ওয়েটিং টাইম হিসাবে একই। তাই গড় রেসপন্স টাইম একটি নির্দিষ্ট প্রসেসের জন্য গৃহীত সময়, যেটি সিপিইউতে এক্সিকিউশন শুরু থেকে রেডি কিউতে ঢোকার সময় বিয়োগফল। উদাহরণস্বরূপ, P 2 এই তাত্ক্ষণিক সময়ে রেডি কিউতে প্রবেশ করে, কিন্তু 7 চক্রের পরেই কেবল CPU চালানো শুরু হয়। সুতরাং, প্রসেস P 2 এর জন্য রেসপন্স টাইম 7 হয়। একইভাবে, প্রসেস P 3 এর একটি রেসপন্স টাইম 11 থাকে কারণ এটি আসলে 11 টি চক্রের জন্য অপেক্ষা করেছে যাতে আসলে CPU এ এক্সিকিউট করা শুরু হয়। সুতরাং, গড়ে, গড় রেসপন্স টাইম 7.75 যা গড় ওয়েটিং টাইমের সমান।

(স্লাইড সময় : 14:07 দেখুন)

এখন, FCFS সিডিউলিং অ্যালগোরিদম একটি চরিত্রগত বৈশিষ্ট্য হল যে সিডিউলিং এর ক্রমটি একটি ব্যাপার। পূর্ববর্তী স্লাইডে অর্থাৎ এই স্লাইডে আমরা অনুমান করলাম যে P1 টি এক্সিকিউট করে তারপর P2 এর এক্সিকিউট করে তারপর P3 এবং তারপর P 4, এবং আমাদের আছে গড় ওয়েটিং টাইম এবং গড় রেসপন্স টাইম 7.75 চক্র। এখন, অনুমান করুন আমরা শুধু ক্রম পরিবর্তন করে বলি, ক্রমানুসারে এখন ক্রমানুসারে P 2 এক্সিকিউট করে, তারপর P 3 এক্সিকিউট করে তারপর P 4 এবং অবশেষে P 1। এই ক্ষেত্রে, আমরা যদি গড় ওয়েটিং টাইম গণনা করি তাহলে আমরা দেখতে পাই এটি 7.75 চক্র থেকে 5.25 চক্র তে কমে যায়। একইভাবে, যদি আপনি গড় রেসপন্স টাইম গণনা করেন, তাহলে আপনি দেখতে পাবেন যে রেসপন্স টাইমে 5.25 চক্র

রয়েছে। একইভাবে, আপনি অন্যান্য মাপকাঠিগুলি গণনা করতে পারেন, যা এখানে উল্লেখ করা হয়েছে এবং আপনি আসলে দুটি অর্ডারিং স্কিমের সাথে পার্থক্য দেখতে সক্ষম হবেন।

(স্লাইড সময় : 15:13 দেখুন)

FCFS সিডিউলিং অ্যালগরিদমের আরেকটি চরিত্রগত বৈশিষ্ট্য হল কনভয় এক্ফেক্ট। মূলত, এই বিশেষ ক্ষেত্রে আমরা দেখতে পারি যে সমস্ত প্রসেস অপেক্ষা করে আছে কেননা একটি বড় প্রসেস সিপিইউকে ধরে আছে। উদাহরণস্বরূপ, এই ক্ষেত্রে, যতক্ষণ না বা যদিও সমস্ত প্রসেস একই সময়ে CPU প্রবেশ করে, সমস্ত প্রসেসগুলিকে শুধুমাত্র P1 এর সমাপ্তির জন্য অপেক্ষা করতে হবে যাতে করে তারা সিডিউলিং হতে পারে। সুতরাং, উদাহরণস্বরূপ আমাদের এখানে একটি বড় প্রসেস থাকলে, আমাদের 7 টি বাস্টের পরিবর্তে প্রসেস P1 এ 100 টি বাস্ট থাকে, অন্য সব প্রসেস P 2, P 3 এবং P 4 100 ব্লকের চক্রের জন্য অপেক্ষা করবে CPU- এ এক্সিকিউট করার জন্য। সুতরাং, এটি একটি FCFS সিডিউলিং পরিকল্পনা যার কিছু খামতি আছে।

(স্লাইড সময় : 16:10 দেখুন)

তবে, FCFS সিডিউলিং অ্যালগরিদম এর বেশ কিছু সুবিধা রয়েছে। প্রথম ব্যাপারটা হল এটি অত্যন্ত সহজ। তাই সিডিউলিং অ্যালগরিদম খুব দ্রুত সম্পন্ন করতে পারে এবং সেইজন্য, সিডিউলিং অ্যালগরিদম দ্বারা গৃহীত সিডিউলিংটি খুব কম হবে, এবং প্রসঙ্গগুলি পরিবর্তন করার সিডিউলিং খুব কম কন্টেক্সট বিলম্বের সাথে শেষ হবে। FCFS সিডিউলিং অ্যালগরিদমের আরেকটি সুবিধা হল এটি ন্যায্য। যতক্ষণ পর্যন্ত কোন প্রসেস সিপিইউতে আসে না, প্রত্যেক প্রসেস অবশেষে চালানো হবে; অথবা অন্য কথায়, যেহেতু প্রত্যেকটি প্রসেস কিছু সময়ের মধ্যে বন্ধ হয়ে যায়, তখনই প্রতিটা রেডি কিউয়ের প্রসেসকেই সিপিইউতে চালানো হবে। এখন আমরা দেখছি যে FCFS সিডিউলারদের দুর্বলতা বা অসুবিধা হল যে ওয়েটিং টাইম আগমনের ক্রমের উপর নির্ভর করে। সুতরাং, আমরা আগের স্লাইডগুলির উদাহরণ দেখছি। আরেকটি অসুবিধা হল যে ছোট প্রসেসটি রেডি কিউতে অপেক্ষা করতে থাকে বড় প্রসেসটি সম্পূর্ণ শেষ না হওয়া পর্যন্ত; বরং, এটি হল কনভয় প্রভাব যেটা আমরা এইমাত্র দেখলাম।

(স্লাইড সময় : 17:19 দেখুন)

এখন, আমাদের আরেকটি সিডিউলিং অ্যালগরিদম দেখবো, যার নাম শর্টেস্ট জব ফার্স্ট সিডিউলিং অ্যালগরিদম। এই নির্দিষ্ট সিডিউলিং অ্যালগরিদমে, জবটি বা প্রসেসটি যার ক্ষুদ্রতম CPU বাস্ট সময় রয়েছে অন্যান্য প্রসেসের আগে সিডিউলিং করা হয়। এখন যদি আপনার একাধিক প্রসেস একই CPU বাস্ট সময়ের সাথে থাকে তবে স্ট্যান্ডার্ড FCFS সিডিউলিংটি ব্যবহার করা হয়। জব ফার্স্ট সিডিউলিং অ্যালগরিদমের দুটি রূপ আছে। প্রথমটি নো প্রিএম্পসন, দ্বিতীয়টি শর্টেস্ট জব ফার্স্ট প্রিএম্পসনের সঙ্গে। এখন SJF উইথ নো প্রিএম্পসন, প্রসেসটি CPU-

বার্স্ট সম্পূর্ণ না হওয়া পর্যন্ত CPU তে এক্সিকিউট করতে থাকবে। প্রিএম্পসনের দ্বিতীয় রূপে, এটি সম্ভব হতে পারে যে প্রসেসটি প্রিএম্পটেড হতে পারে, যখন একটি নতুন প্রসেস রেডি কিউতে আসে। আমরা পরে আরও দেখব, কিন্তু প্রথমে আমরা শর্টেস্ট জব ফার্স্ট উইথ নো প্রিএম্পসনটি দেখবো।

(স্লাইড সময় : 18:21 দেখুন)

তাই আসুন আমরা আগের উদাহরণটিই দেখি, আমাদের এখানে চারটি প্রসেস আছে P 1 থেকে P 4 এবং তাদের সবগুলি তাত্ক্ষণিকভাবে 0 তে এসে পৌঁছয়, অর্থাৎ এই 0 সময়ের মুহূর্তে এই চারটি প্রসেস P 1 থেকে P 4 রেডি কিউতে প্রবেশ করে। এবং এই প্রসেসের প্রতিটির একটি পৃথক CPU বার্স্ট টাইম আছে যথাক্রমে 7, 4, 2 এবং 1। এখন প্রথমে, সিপিডি সিডিউলার বিভিন্ন বার্স্টের সময় দেখবে এবং ন্যূনতমটি খুঁজে পাবে। সুতরাং, এই ক্ষেত্রে আমরা P 4 সর্বনিম্ন CPU বার্স্ট সময় আছে দেখুন, তাই এটিই প্রথম সিডিউল হবে। সুতরাং, প্রসেসটি সম্পূর্ণ না হওয়া পর্যন্ত P 4 সিডিউল হয়, এই বিশেষ ক্ষেত্রে এটি 1 চক্রের মধ্যে সম্পন্ন হয়। তারপর অবশিষ্ট তিনটির মধ্যে, আমরা দেখছি যে P 3 সর্বনিম্ন CPU বার্স্ট সময় আছে। সুতরাং, P 3 প্রসেসটি সিডিউল হয় এবং সম্পূর্ণ বার্স্ট পর্যন্ত এক্সিকিউট হয়। তারপর P 2 সিডিউল হয় কারণ এটির বার্স্ট টাইম 4, যখন P1 এর একটি উচ্চতর বার্স্ট 7 রয়েছে। এবং পরিশেষে, P 1 সম্পূর্ণ সমাপ্তি পর্যন্ত চালানো হবে। এখন, গড় ওয়েট টাইম, যদি আমরা গণনা করি তবে তা হয় 2.75, গড় রেসপন্স টাইমও 2.75 যেমন ছিল ওয়েট টাইমের ক্ষেত্রে।

(স্লাইড সময় : 19:43 দেখুন)

এখন আসুন আমরা আরেকটি উদাহরণ দেখি শর্টেস্ট জব ফার্স্ট উইদাউট প্রিএম্পসনের। তাই, আমরা একই চারটি প্রসেস P1 থেকে P 4 পর্যন্ত নেব এবং প্রতিটি প্রসেসই আগের মত যথাক্রমে 7, 4, 2 এবং 1 একই বার্স্ট টাইম আছে। যাইহোক, তারা বিভিন্ন মুহূর্তে রেডি কিউতে প্রবেশ করে। তাই, P 1 0 মুহূর্তে প্রবেশ করে, দ্বিতীয় মুহূর্তে P 2, P 3 চতুর্থ মুহূর্তে এবং সপ্তম মুহূর্তে P 4 প্রবেশ করে। এখন, এটি ন্যাট চার্টে একটি সামান্য পরিবর্তন, যেখানে দেখানো হয়েছে যে CPU- এ কোন প্রসেসটি চালানো হচ্ছে, এটি প্রসেসগুলো যে ক্রমে উপস্থিত হয় তাও প্রদর্শন করে। এটি দেখায় যে P1 প্রথম আসে, তারপর P 2 দ্বিতীয়, তারপর P 3 এবং অবশেষে P 4 সপ্তম মুহূর্তে আসে। এখন, যখন এই নির্দিষ্ট মুহূর্তে যখন সিডিউলারটি এক্সিকিউট আরম্ভ করে, তখন এই বিশেষ বিন্দুতে এসে পৌঁছানো একমাত্র প্রসেস হল P 1 তাই এটি চালানোর জন্য P1 সিডিউলিং করে। সুতরাং, P 1 তার সমগ্র বার্স্টের জন্য সংশ্লিষ্ট হয় যা 7 চক্রের, এবং তারপর এই নির্দিষ্ট চক্রের মধ্যে, পুনরায় সিডিউলার প্রবেশ করে বা আবার একটি সিডিউলার এক্সিকিউট করে আর এটা তিনটি প্রসেস পেয়েছে P 2, P 3 আর P 4 কে রেডি কিউ থেকে নির্বাচন করবে। এবং তাদের মধ্যে থেকে P 4 এর সংক্ষিপ্ততম বার্স্টের সিডিউলিং তাই এটি এক্সিকিউটের জন্য নির্বাচিত হয়। অতএব, P 4 সিপিইউতে এক্সিকিউট করে, এবং তারপর P 3 কারণ P 3 P2 এর তুলনায় একটি

সংক্ষিপ্ত বাস্ট টাইম আছে এবং পরিশেষে, P 2 এক্সিকিউট হয়। সুতরাং, যদি আমরা গড় ওয়েটিং টাইম গণনা করি, আমরা দেখতে পাই যে এটি 3 চক্র।

(স্লাইড সময় : 21:37 দেখুন)

অতএব, শর্টেস্ট জব ফার্স্ট সিডিউলিং অ্যালগরিদম এর সুবিধা হল এটি সর্বোত্তম। এটি সর্বদা সর্বনিম্ন গড় ওয়েটিং টাইম অপেক্ষা করবে। এবং এর ফলে, গড় রেসপন্স টাইমও হ্রাস পায়। SJF সিডিউলিং অ্যালগরিদমের প্রধান অসুবিধা যে এটি ব্যবহারিক নয়; মূলত, বাস্টের সময় কী হবে তা ভবিষ্যদ্বাণী করা খুব কঠিন। সুতরাং SJF সিডিউলিং অ্যালগরিদমের আরেকটি দুর্বলতা হল যে কিছু কাজ স্টার্ড হতে পারে; মূলত যদি আপনার একটি প্রসেস থাকে যা একটি অত্যন্ত দীর্ঘ CPU বাস্ট টাইম আছে, তাহলে এটি আসলে CPU- এ চালানোর সুযোগ নাও পেতে পারে।

(স্লাইড সময় : 22:19 দেখুন)

এখন, আমরা শর্টেস্ট জব ফার্স্ট সিডিউলিং অ্যালগরিদম (SJF) উইথ প্রিএম্পসন দেখবো। এটিও শর্টেস্ট রিমেইনিং টাইম ফার্স্ট। তাই এই অ্যালগরিদমের মৌলিক ধারণা হল যে যদি একটি নতুন প্রসেস রেডি কিউতে আসে, এবং এই প্রসেসের বর্তমান অবশিষ্ট প্রসেসগুলোর চেয়ে কম বাস্ট টাইম আছে, তারপর একটি কন্ট্রোল সুইচ হয় এবং নতুন প্রসেস CPU মধ্যে সিডিউলিং হয়। এটা আবার গড় ওয়েটিং টাইম এবং সেইসাথে গড় রেসপন্স টাইম হ্রাস করে। যাইহোক, আগের ক্ষেত্রে যেমন শর্টেস্ট জব ফার্স্ট উইথ নো প্রিএম্পসন এখানেও বাস্তব নয়। সুতরাং আসুন একটি উদাহরণ সঙ্গে আরো বুঝি।

(স্লাইড সময় : 23:02 দেখুন)

সুতরাং আসুন আমরা একই উদাহরণ গ্রহণ করি যেখানে 4 টি প্রসেস আছে যাদের বাস্ট টাইম 7, 4, 2 এবং 1; এবং 0,2, 4 এবং 7 হল আগমনের সময়, ন্যাট চার্টকে টাইম প্রসেস হিসাবে বিকশিত করবে। সুতরাং, 0 মুহূর্তে, বর্তমান প্রসেস উপস্থিত যা P1; এবং সেইজন্য, সিডিউলিংকারীর কোন পছন্দ নেই, কিন্তু CPU- এ P 1 কে সিডিউল করে।এরপে P 1 2 ঘড়ি চক্রের জন্য চালানো হবে। এখন দ্বিতীয় ঘড়ি চক্র পরে প্রসেস P 2 রেডি কিউতে প্রবেশ করেছে। এখন আপনি দেখতে পান যে P 2 4 চক্রের একটি বাস্ট আছে। যাইহোক, P 1 এর 5 টি চক্রের অবশিষ্ট বাস্ট রয়েছে। সুতরাং, এই দ্বারা কি বোঝানো হয় যে 7 চক্রের সিপিইউ বাস্টের সিডিউলিং, P 1 2 চক্র সম্পন্ন হয়েছে সুতরাং, থাকে 5। এখন সিডিউলার P1 এর 5 চক্রটি দেখতে পারে, যা P 2 এর চেয়ে বড়, যা 4 চক্রের বাস্ট ঘটেছে। অতএব, এটি একটি কন্ট্রোল সুইচ করবে এবং চালানোর জন্য P 2 কে সিডিউল করবে। তাই, P 2 2 ঘড়ির চক্রের জন্য চালানো হবে এবং তারপর P 3 চতুর্থ ঘড়ি চক্রে আসবে। তাই, এই বিশেষ ক্ষেত্রে, সিডিউলারটি জানতে পারবে যে P 3 এর দুটি চক্রের বাস্টের সিডিউলিং রয়েছে, আর P 2 এর দুটি চক্রের

অবশিষ্ট বাস্টের সিডিউলিং রয়েছে; আমাদের 2 আছে কারণ P 2 এর 4 চক্র বাস্ট টাইম ছিল যার মধ্যে 2 টি সম্পন্ন হয়েছে। সুতরাং, আরো 2 টি চক্র অবশিষ্ট। যেহেতু P 2 পুরাতন প্রসেসটি CPU-র উপর চলছে সেহেতু এটি অবশিষ্ট 2 চক্র বাস্ট টাইম, এবং P 3-এর নতুন প্রসেসে দুটি চক্রের বাস্ট টাইম আছে, তাই কোন প্রিএম্পসন নেই এবং P 2 চালানো অব্যাহত থাকবে। এখন P 2 সম্পূর্ণ হওয়ার পরে, এই বিশেষ ক্ষেত্রে, P 3 3 চক্রের জন্য সঞ্চালন করে, এবং তারপর P 4 এখন প্রবেশ করে, যার পরে আপনি যাচাই করবেন যে কোনও প্রিএম্পটেডের কারণ হবে না। তাই, P 3 সমাপ্ত হওয়ার পর, সিডিউলার P4 বা P1 চালানোর সিদ্ধান্ত নেয়। তাই, আমরা দেখি যে P 4 এর একটি বাস্ট সাইকেল আছে, যখন P 1 এর অবশিষ্ট বাস্টচক্র 5 এর থাকে, তাই সিডিউলিং সিডিউলার সিদ্ধান্ত নেবে P 4 ওভার P 1। তাই, P 4 সিপিইউতে রান করে এবং এটি সমাপ্ত হলে P 1 এর অবশিষ্ট বাস্টের সিডিউলিং চালানো হবে। তাই, যদি আপনি গড় ওয়েটিং টাইমের গণনা করেন, তবে আপনি দেখতে পাবেন যে এটি 2.5 এ নেমে এসেছে, যখন গড় রেসপন্স টাইম 0.75 তে হ্রাস পায়। যাইহোক, যেহেতু আমরা আগেই উল্লেখ করেছি যে, শর্টেস্ট জব ফাস্ট এর মত, এই সিডিউলিং অ্যালগরিদম প্রথাটিও বাস্তবায়নের জন্য কার্যকর নয়, কারণ এটি প্রসেসের বাস্ট টাইম কত এবং আরো কঠিন এটা সনাক্ত করা যে কত বাস্ট টাইম অবশিষ্ট আছে।

(স্লাইড সময় : 26:08 দেখুন)

এখন আমরা অন্য একটি সিডিউলিং অ্যালগরিদম দেখব যা রাউন্ড রবিন সিডিউলিং অ্যালগরিদম হিসাবে পরিচিত। সুতরাং, মূলত রাউন্ড রবিন সিডিউলিং অ্যালগরিদমে, একটি প্রসেস একটি সিডিউলিং স্লাইসের জন্য সঞ্চালিত হয় অর্থাৎ একটি প্রসেস একটি সিডিউলিং স্লাইসে সঞ্চালিত হয় এবং যখন সিডিউলিং স্লাইসটি সম্পূর্ণ করে তখন এটি রেডি কিউতে সরানো হয়। সুতরাং, এই রাউন্ড রবিন সিডিউলিং অ্যালগরিদম অর্জন করার জন্য যা একটি প্রিএম্পটিভ সিডিউলিং অ্যালগরিদম; এখন রাউন্ড রবিন সিডিউলিং অর্জন করার জন্য, আমরা নিয়মিতভাবে CPU কে ইন্টারাপ্ট করার জন্য সিস্টেমের টাইমার কনফিগার করতে হবে। প্রত্যেক টাইমারের বিরতিতে, কার্নেলটি বর্তমান প্রসেসকে প্রত্যাহার করবে এবং সিপিইউতে চালানোর জন্য আরেকটি প্রসেস পছন্দ করবে।

(স্লাইড সময় : 26:50 দেখুন)

আসুন একটি উদাহরণ সঙ্গে রাউন্ড রবিন সিডিউলিং অ্যালগরিদম আলোচনা করি। তাই, আমরা যেসব সিডিউলিং অ্যালগরিদমগুলি দেখেছি তার সাথে এক পার্থক্য হল, সিডিউলিং স্লাইসের ধারণা। সুতরাং, এটি ন্যাট চার্ট। তাই, বিশেষত এই ক্ষেত্রে, এই সময়ের মধ্যে, 2-এর সমান সিডিউলিং ধরে দেখুন, আমরা 2-এর মতো সময়কে স্লাইস রেখেছি, একটি টাইমার ইন্টারাপ্ট আছে এবং টাইমার এর ইন্টারাপ্টের ফলে সিডিউলিংটি শুরু হবে এবং সম্ভাব্য অন্য একটি প্রসেস CPU-র মধ্যে সিডিউল হচ্ছে। সুতরাং, একটি ডাটা স্ট্রাকচার যা রাউন্ড

রবিনের সিডিউলিং আলগোরিদিম বাস্তবায়নে খুবই উপযোগী টা হল ফিফো। এই নির্দিষ্ট FIFO পরবর্তী কার্যকর করা প্রসেসগুলিকে CPU র মধ্যে সঞ্চয় করে। উদাহরণস্বরূপ, এই বিশেষ ক্ষেত্রে, P 2 ফিফোর শীর্ষে রয়েছে, তাই এটি একটি পরবর্তী প্রসেস যা ফীফোতে চালানো হয়। সুতরাং, P 2 এখানে এক্সিকিউট করে। সুতরাং, উদাহরণস্বরূপ, আমরা এখনও 4 টি প্রসেস বিবেচনা করব যেমনটি আমরা P1 থেকে P4 পর্যন্ত করেছি এবং আমরা অনুমান করব যে তাদের সবগুলি 0 মূহুর্তে এসে পৌঁছায় এবং তারা রেডি কিউটির ফিফোতে প্রবেশ করে এবং তাদের বার্ট টাইম হয় 5, 4, 2 এবং 3 যথাক্রমে। সুতরাং আসুন আমরা আলোচনার জন্য বলি যে সিডিউলার এই বিশেষ অর্ডার P1, P 2, P 3 এবং P 4 দিয়ে শুরু করে এবং এটি প্রথমে P 1 চালানো পছন্দ করে। সুতরাং, P 1 2 চক্রের জন্য সঞ্চালিত হয় এবং তারপর একটি ইন্টারাপ্ট হয় ফলে কন্ট্রোল সুইচ ঘটে এবং তারপর এই বিশেষ ক্ষেত্রে FIFO শীর্ষের P 2 সিপিইউতে সিডিউলিং হয়, যখন P 1টি ফিফোতে প্রবেশ করে। তাই, P 2 তারপর পরবর্তী টাইমার ইন্টারাপ্ট পর্যন্ত 2 টি চক্রের জন্য সঞ্চালিত হয়, যখন এই ক্ষেত্রে 2 চক্রের টাইম স্লাইস সমাপ্ত হয় এবং তারপর এটি ফিফোতে প্রবেশ করে। তাই, P 2 এখন ফিফোর একেবারে নীচে, আর P 3 একেবারে উপরে তাই P 3 রান করার জন্য সিডিউল হয়। সুতরাং, এই ভাবে, প্রতি দুই চক্রে একটি নতুন প্রসেস CPU মধ্যে সিডিউল হয় এবং এক্সিকিউট হতে পারে। সুতরাং, যদি আমরা গড় ওয়েটিং টাইম গণনা করি। সুতরাং, এই বিশেষ ক্ষেত্রে, আমরা দেখব যে গড় ওয়েটিং টাইম এবং গড় রেসপন্স টাইম ভিন্ন। সুতরাং, P 1এর গড় ওয়েটিং টাইম কি? সুতরাং, P 1 এখানে 2 টি চক্র চালানো হয়, এখানে 2 টি চক্র এবং এখানে এক্সিকিউসন সমাপ্ত করে। সুতরাং, এটি চক্রের বাকি অংশে রেডি কিউতে অপেক্ষা করছে। তাই অপেক্ষা করছে চক্র সংখ্যা 1, 2, 3, 4, 5, 6, 7, 8, 9। সুতরাং, P 1 9 চক্রের জন্য অপেক্ষা করে; এখন P 2 8 চক্রের জন্য অপেক্ষা করছে - 1, 2, 3, 4, 5, 6, 7, 8; 4 টি চক্রের জন্য P 3 এবং 10 টি চক্রের জন্য P 4 সুতরাং গড় ওয়েটিং টাইম হল 7.75 চক্র। এখন, গড় রেসপন্স টাইম গণনা করতে আগে আমরা এটি আগেই সংজ্ঞায়িত করেছি, রেসপন্স টাইম হল সেই টাইম প্রসেসটি রেডি কিউতে প্রবেশ করে তখন থেকে CPU তে এক্সিকিউট শুরু করার সময় পর্যন্ত, সেই ল্যাটেন্সিটি রেসপন্স টাইম হবে। উদাহরণস্বরূপ, P 1 এর একটি রেসপন্স টাইম হল 0 আছে কারণ এটি রেডি কিউতে প্রবেশ করে বা FIFO তে প্রবেশ করে এবং অবিলম্বে এক্সিকিউসন শুরু করে। P-2 অপরদিকে 0 মূহুর্তে প্রবেশ করলেও 2 চক্রের পরেই চালানো হয়, তাই এটির রেসপন্স টাইম 2 হয়। অনুরূপভাবে, P 3 ও 0 মূহুর্তে প্রবেশ করে কিন্তু শুধুমাত্র এই পয়েন্টে এক্সিকিউট হয়। তাই, এ মূহুর্তে এই তাত্ক্ষণিক সময়ে, এটির রেসপন্স টাইম হয় 4 এবং P 4 6 এর একটি রেসপন্স টাইম আছে সুতরাং, গড় রেসপন্স টাইম 3 হয়। এখন মোট 7 টি কন্ট্রোল সুইচ হয়। সুতরাং, এই ক্ষেত্রে , 1, 2, 3, 4, 5, 6 এবং একটি কন্ট্রোল সুইচ এখানে দেখা যায় কারণ প্রসেস P 4টি বিদ্যমান এবং P 1টি এক্সিকিউট হতে থাকে। তাই এখানে কন্ট্রোল সুইচের সংখ্যা হল 7, যা 1, 2, 3, 4, 5, 6 এবং সপ্তম কন্ট্রোল সুইচ এখানে দেখা যায় তারপর P 4 প্রস্থান করে এবং P 1টি CPU এ সুইচ করে।

(স্লাইড সময় : 31:42 দেখুন)

এখন আসুন রাউন্ড রবীন সিডিউলিংএর আরেকটি আরও জটিল উদাহরণ দেখি, যেখানে আমাদের আগমনের সিডিউলিংও এক নয়। সুতরাং, P 1 টি 0 মূহুর্তে উপস্থিত হয়; দ্বিতীয় P 2, P 3 এবং P 4 যথাক্রমে তৃতীয় ও নবম স্থানে। সুতরাং, আমরা একইভাবে এই ঘটনার জন্য ন্যাট চার্ট এবং ফিফোর স্টেট আঁকতে পারি। তাই 0 মূহুর্তে শুরু করি, একমাত্র প্রসেস P1 এসেছে এবং অতএব, P 1 2 চক্রের জন্য এক্সিকিউট হবে। এবং এই বিশেষ সময়ে, যখন টাইমার ইন্টারাপ্ট সংঘটিত হয়, অন্য কোন প্রসেস এখনও উপস্থিত হয় নি, সুতরাং P1 অন্য সিডিউলিং চক্রের জন্য আরও 2 টি চক্র এক্সিকিউটসন চালিয়ে যাবে। যাইহোক, এই সিডিউলিং স্লাইসে, আমাদের দুটি প্রসেস যা রেডি কিউতে আছে; এই প্রসেসটি P 2 এবং P 3 হয়। সুতরাং, P 2 এই ব্যবধান সময়ে আসে যখন P 3 এই ব্যবধানে আসে এবং তারা ফিফোতে যুক্ত হয়। সুতরাং, দ্বিতীয়বার স্লাইস সমাপ্তিতে, একটি কন্ট্রোল সুইচ হয় এবং P 2 এক্সিকিউটের জন্য CPU-র মধ্যে সিডিউল হয় যখন P 1 নির্বাহ করা হয় যা তারপর ফিফোর মধ্যে যায়। সুতরাং, P 2 2 চক্রের জন্য এক্সিকিউট হয়, তারপর P 3 2 চক্রের জন্য এক্সিকিউট হয়, তারপর P 1 2 চক্রের জন্য এক্সিকিউট এবং সেই সময়ে P 4 এসে এবং FIFO-এ যুক্ত হয়ে যায়। এখন, আমাদের তিনটি প্রসেস P 1, P 2 এবং P 4 আছে এবং এইগুলি সময়কালের জন্য চালানোর সিডিউল পায়। তাই এই ক্ষেত্রে গড় ওয়েটিং টাইম 4.75 হয়, গড় রেসপন্স টাইম 2 হয়। তাই, কিভাবে গড় ওয়েটিং টাইম 4.75, এটি প্রসেসটি P 1 7 চক্র জন্য অপেক্ষা করে এটি সমাপ্ত করার আগে। সুতরাং যে 1, 2, 3, 4, 5, 6 এবং 7। প্রসেস P 2 6 চক্রের জন্য অপেক্ষা করেছে, তাই এটি 1, 2, 3, 4, 5, 6; প্রসেস 3 3 চক্রের জন্য অপেক্ষা করেছে, এবং প্রসেস 4 3 চক্রের জন্য অপেক্ষা করেছে। গড় ওয়েটিং টাইম 2 এর সমান হতে পারে এবং কনটেন্ট সুইচ সংখ্যা আগের মত 7 হয়।

(স্লাইড সময় : 34:15 দেখুন)

এখন আসুন আমরা আগে যেমন করেছি আমরা একই উদাহরণ নিতে পারি, কিন্তু আমরা টাইম স্লাইস 2 থেকে হ্রাস করে 1 করেছি। সুতরাং, আমরা দেখছি যে প্রত্যেক মূহুর্তে আমাদের একটি টাইমার ইন্টারাপ্ট এবং সম্ভাব্য কন্ট্রোল সুইচ ঘটতে পারে। যদি আমরা এই বিশেষ ক্ষেত্রে গড় ওয়েটিং টাইম এবং গড় রেসপন্স টাইম গণনা করি তবে আমরা দেখি যে বিশেষভাবে গড় রেসপন্স টাইম কমেছে। কেন এই ঘটেছে কারণ একবার একটি প্রসেস কিউতে করলে এটি কম চক্রের জন্য অপেক্ষা করতে হয় এটিকে CPU মধ্যে সিডিউলিং করার জন্য। অন্যদিকে, কন্ট্রোল সুইচগুলির সংখ্যা 7 থেকে 11 পর্যন্ত বৃদ্ধি পেয়েছে। যেহেতু আমরা টাইমার ইন্টারাপ্টগুলি যা বেশি ঘন ঘন তাই নিয়েছি, তাই সেখানে একটি কনট্রোল সুইচ ঘটবে এমন সম্ভাবনা বেশি।

(স্লাইড সময় : 35:10 দেখুন)

এখন, যদি আমরা একই উদাহরণ গ্রহণ করি, তবে 1 এর পরিবর্তে 5 এর সিডিউলিং স্লাইসের সাথে এবং যদি আমরা গড় ওয়েটিং টাইম এবং গড় রেসপন্স টাইম গণনা করি, তাহলে আমরা দেখতে পাব যে রেসপন্স টাইম যথেষ্ট বৃদ্ধি পায় 2.75 পর্যন্ত যখন কনটেন্ট সুইচ সংখ্যা 4 থেকে অনেকটা হ্রাস করে। অন্যদিকে, আমরা দেখি যে সিডিউলিংটি ফার্স্ট কাম ফার্স্ট সার্ভ এর মত আচরণ করা শুরু করে। রেসপন্স টাইম খারাপ কারণ বড় টাইম স্লাইস সিডিউলিংটি আরো এবং আরো ফার্স্ট কাম ফার্স্ট সার্ভ এর মতো আচরণ করবে যা আমরা জানি একটি খারাপ রেসপন্স টাইম। তাই, এই সব উদাহরণ যে আমরা এতক্ষণ দেখেছি আমরা উপসংহার করতে পারি যে একটি টাইম স্লাইসের ব্যাপ্তি অত্যন্ত জটিল। সুতরাং, এটি উভয়ই রেসপন্স টাইম আর পাশাপাশি কন্টেক্সট সুইচ সংখ্যার প্রভাব।

(স্লাইড সময় : 36:07 দেখুন)

সুতরাং, মূলত, যদি আমাদের একটি খুব সামান্য কোয়ান্টামের টাইম স্লাইস থাকে, তবে সুবিধা হচ্ছে যেগুলি CPU- র মধ্যে সিডিউলিং করার আগে প্রসেসগুলিকে রেডি কিউতে খুব বেশি অপেক্ষা করতে হবে না। মূলত এই এটা বোঝায় যে প্রসেসের রেসপন্স টাইম খুব ভাল হবে বা এটির কম রেসপন্স টাইম হবে। অন্যদিকে, অল্প টাইম স্লাইস আবার খারাপ, কারণ আমরা খুব ঘন ঘন কনটেক্সট সুইচ করতাম। এবং আমরা আগে দেখেছি যে কন্টেক্সট সুইচের উল্লেখযোগ্য ওভারহেডস হতে পারে। অতএব, এটি সিস্টেমের কর্মক্ষমতা হ্রাস করে। দীর্ঘমেয়াদি স্লাইস বা দীর্ঘ কোয়ান্টামের একটি অসুবিধা আছে যা প্রসেস একসঙ্গে চালানোর জন্য প্রসেসকৃত হয় না, এটি দেখায় ফার্স্ট কাম ফার্স্ট সার্ভ এর মত সিডিউলিং অ্যালগোরিদম এবং এর ফলে এটি আবার সিস্টেমের পারফরম্যান্স হ্রাস করে। সুতরাং, সাধারণত একটি আধুনিক দিনের অপারেটিং সিস্টেমের টীমে স্লাইসের মেয়াদ 10 মিলিসেকেন্ড থেকে 100 মিলিসেকেন্ডে রাখা হয়। সুতরাং, xv6 প্রোগ্রাম, প্রোগ্রাম টাইমার প্রতি 10 মিলিসেকেন্ডের মধ্যে ইন্টারাপ্ট করে।

(স্লাইড সময় : 37:17 দেখুন)

সুতরাং রাউন্ড রবিন নির্ধারণের অ্যালগরিদমটি নিম্নরূপ: অ্যালগরিদমটি ন্যায্য কারণ প্রতিটি প্রসেস সিপিইউতে চালানোর একটি ন্যায্য সুযোগ পায়। গড় ওয়েটিং টাইম বিশেষ করে যখন বার্ট টাইম আলাদা এবং রেসপন্স টাইম খুব ভাল। অন্যদিকে, রাউন্ড রবিনের সিডিউলিং অ্যালগরিদমের দুর্বলতা নিম্নরূপ: কন্টেক্সট সুইচিং এর সংখ্যা বৃদ্ধি এবং যেহেতু আমরা কন্টেক্সট সুইচিং এর উল্লেখযোগ্য বাধগুলি আগে দেখেছি। এবং দ্বিতীয় অসুবিধা হল যে গড় ওয়েটিং টাইম উচ্চ বিশেষ করে যখন বার্টের টাইম সমান দৈর্ঘ্যের হয়।

(স্লাইড সময় : 37:56 দেখুন)

xv6 সিডিউলিং নীতি রাউন্ড রবীন সিডিউলিং এর নীতির একটি বৈকল্পিক; সোর্স কোড এখানে দেখানো হয়। সুতরাং, মূলত কি xv6 সিডিউলিংটি এটি ptable অ্যারের মাধ্যমে পড়ছে। সুতরাং, আমরা আগে ptable দেখেছি, যা এই বিশেষ অ্যারের মাধ্যমে procs এবং সিডিউলার পার্সেস পরের প্রসেস খুঁজে বের করে যা রানলেবল এবং সুইচকে ইনভোক করবে। সুতরাং, এই হল যেখানে আপনি কন্টেক্সট সুইচ আহ্বান করেন। সুতরাং, প্রত্যেক সময় সিডিউলার সঞ্চালিত হয়, এই অ্যারের মধ্যের পরবর্তী প্রসেস যেটি রানেবেল তার সিডিউলিং হয়। পরবর্তী, আমরা অগ্রাধিকারের উপর ভিত্তি করে সিডিউলিং এলগরিদম দেখব। সুতরাং, এই অগ্রাধিকার ভিত্তিক সিডিউলিং অ্যালগরিদম। ধন্যবাদ।